

LangGraph AI 에이전트 개발_2일차

한 기 영

(주)데이터인사이트

과정 개요

- ✓ RAG 개념 이해
- ✓ 벡터DB 구축
- ✓ AI Agent with RAG
- ✓ 종합실습 : 고객 서비스 Agent

RAG 개념 이해

RAG Retrieval Augmented Generation

✓ 대규모 텍스트 데이터 및 임베딩 벡터를 저장, 검색용

✓ LLM with RAG 절차 다시 살펴보기

- ① 사용자 질문을 받음
 - 임베딩 : 벡터로 변환(질문 벡터)
- ② **준비된 정보 DB**에서 답변에 필요한 문서 검색
 - [질문 벡터]와 DB내 저장된 [문서 벡터]와 유사도 계산
 - 가장 유사도가 높은 문서 n개 찾기
- ③ 필요한 문서를 포함한 프롬프트 생성
- ④ LLM이 답변 생성하기

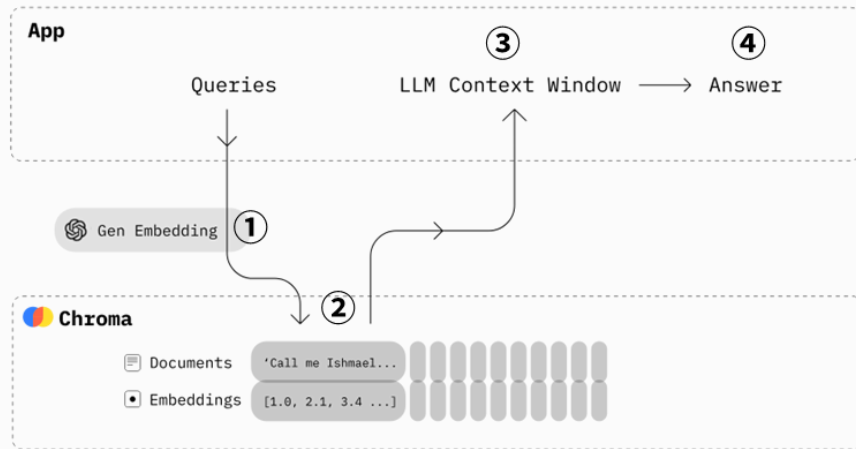
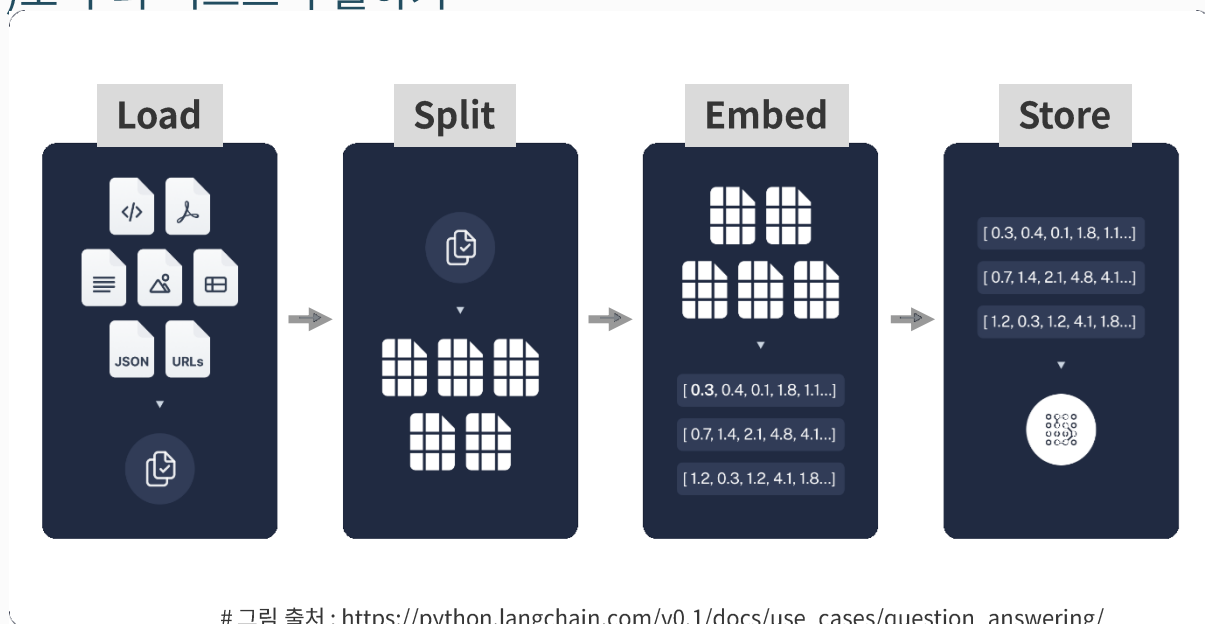


그림 출처 : <https://docs.trychroma.com/>

Vector DB 구축 절차

✓ Vector DB 구축 절차

- 텍스트 추출 : Loader
 - 다양한 문서(word, pdf, web page 등)로 부터 텍스트 추출하기
- 텍스트 분할 : Splitter
 - chunk 단위로 분할
 - Document 객체로 만들기
- 텍스트 벡터화 : Text Embedding
- Vector DB로 저장 : Vector Store



Loader

✓ 다양한 소스에서 문서를 불러와 처리하는 모듈

다양한 소스 지원

- 웹 페이지, PDF, DB 등 여러 형식의 문서 로딩 가능

데이터 변환 및 정제

- 후속 처리에 적합한 형태로 변환하고 불필요한 데이터 제거

효율적인 데이터 관리

- 대량 문서를 빠르게 접근·검색 가능한 구조로 저장

Splitter

✓ 긴 문서를 의미 단위의 청크(chunk)로 분할하는 도구

- 청킹하는 이유

- LLM은 입력 토큰 수에 제한이 있음.
- 지나치게 긴 텍스트는 핵심 정보 외 노이즈를 포함 → RAG 품질 저하

✓ 분할 시 고려 사항

텍스트 분할 의미

- 문장·구절·단락 등 의미가 독립적으로 유지되는 단위로 분할

청크 크기

- LLM 입력 한도·비용을 고려해 단어 수·문자 수 기준으로 설정

Embedding

✓ 임베딩

- 텍스트 데이터를 숫자로 이루어진 벡터로 변환하는 과정.
- 이때, 의미적인 정보를 보존하도록 설계

✓ 임베딩 목적

- 벡터 표현을 사용하면, 텍스트 데이터를 벡터 공간 내에서 수학적으로 다룰 수 있게 됨
- 이를 통해 텍스트 간의 유사성을 계산 가능

✓ 주요 모델

- Openai : text-embedding-3-small
- Microsoft : e5-large-v2

Vector Store

✓ 벡터 저장소(Vector Store)

- 임베딩 벡터를 저장하고 유사도 기반으로 빠르게 검색하는 DB (Vector DB)
- 대표적인 Vector DB : Chroma, FAISS

구분	Chroma	FAISS
개발사	Chroma (오픈소스)	Meta/Facebook (오픈소스)
종류	DB 솔루션	벡터 검색 라이브러리
영속성(Persistence)	지원	기본 미지원 (직접 구현 필요)
메타데이터 필터링	지원	미지원
GPU 가속	미지원	지원
적합한 규모	중소규모	수억 ~ 수십억 개 대규모 벡터
사용 난이도	쉬움 (간단한 API)	상대적으로 복잡
LangChain 연동	매우 간단	가능

요약 | Overview

✓ RAG : 지식 DB를 기반으로 LLM이 답변하는 방식

✓ 벡터 DB

- 절차 : 지식(문서) 파일 로딩 ➡ 청킹 ➡ 임베딩 & 저장
- 청킹
 - LLM 모델의 입력 토큰의 개수가 정해져 있기 때문
 - 텍스트가 너무 긴 경우에는 핵심 정보 이외에 불필요한 정보들이 많이 포함 ➡ RAG 품질 저하 요인
 - 핵심 정보가 유지될 수 있는 적절한 크기로 나누는 것이 매우 중요
- 임베딩 : 텍스트 데이터를 숫자로 이루어진 벡터로 변환
- 저장 : 벡터DB를 지원하는 DBMS에 저장(Chroma, FAISS 등)

벡터 DB 구축

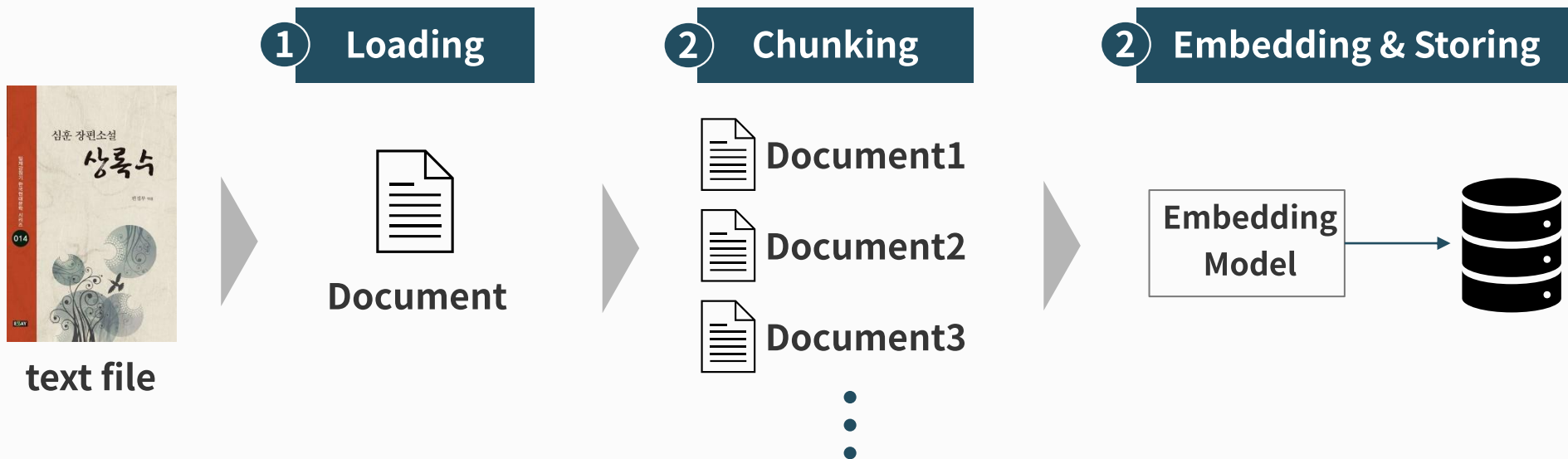
| 간단한 벡터 DB 구축하기

간단한 예제로 벡터DB를 구현해 봅시다.

✓ 예제

- 상록수 책으로 벡터 DB를 만들어 봅시다.

✓ 단계



① Load : txt 파일 로딩

✓ TextLoader

- txt 파일을 읽고, 문서 객체(Document)로 변환
 - txt 파일 로딩 시 1개의 Document 로 저장

```
from langchain_community.document_loaders import TextLoader

# 텍스트 파일 경로 지정
file_path = "상록수.txt"

# TextLoader를 이용하여 문서 로드
loader = TextLoader(path + file_path)
txt_docs = loader.load()

# 로드된 문서 출력
print(txt_docs)
```

```
[Document(metadata={'source': '/content/drive/MyDrive/ai_agent/상록수.txt'},
page_content='심훈 <상록수>\n 쌍두취 행진곡 가을...
```

① Load : txt 파일 로딩

✓ Document 객체

- LangChain에서 텍스트 데이터(문서)를 구조적으로 표현하는 기본 단위
- 주요 속성
 - **metadata** : 문서의 출처, 태그, 카테고리 등의 부가 정보
 - **page_content** : 문서의 실제 텍스트 내용
- Loader로 로딩한 후 저장하거나, 직접 Document 생성 가능

```
from langchain_core.documents import Document

doc = Document( page_content="이것은 LangChain의 Document 객체 예제입니다.",
                metadata={"source": "sample.txt", "category": "example"} )
```

② Split : 청킹

✓ 청킹 기준 고려사항

- LLM 모델의 입력 크기(토큰 수)와 비용
- 문서의 종류, 성격, 내용구성

✓ 예제 : 상록수 책을 나눈다면 어떻게 나누는 것이 좋을까요?

- 문자 수, 단어 수 단위 : 기계적으로 문서를 자름.
- 문단 단위 : 문맥이 자연스럽게 연결될 수 있도록 문단을 기준으로 나누기
- 대화 단위 : 소설의 특성상 인물 간 대화가 많으므로, 대화가 끊어지는 부분에서 나누기
- 내용 전환 시점: 장면이 바뀌는 부분에서 적절히 분리

② Split : 청킹

✓ CharacterTextSplitter

- chunk_size : 청크의 최대 글자수
- chunk_overlap : 겹치는 글자수
- separator : 구분자
 - ' ': 글자 단위
 - '.': 문장단위
 - '\n': 줄 바꿈 단위
 - '\n\n': 단락 단위

```
from langchain_text_splitters import CharacterTextSplitter

text_splitter = CharacterTextSplitter(
    chunk_size = 500,
    chunk_overlap = 100,
    separator = '\n', # \n 줄바꿈 문자 기준으로 자르기,
)

split_texts = text_splitter.split_documents(txt_docs)
```

✓ text_splitter

구분	설명	결과 형태
.split_documents	Document 분할 시 사용.	List[Document]
.split_text	텍스트 분할 시 사용	List[str]

② Split : 청킹

✓ CharacterTextSplitter

- 청크 단위로 자르다 보면 다음과 같은 경고 문구 발생될 수 있음.

```
WARNING:langchain_text_splitters.base:Created a chunk of size 612,  
which is longer than the specified 500
```

- CharacterTextSplitter가 chunk_size=500으로 설정돼 있음에도 불구하고, 612자짜리 청크를 만들어버렸다는 뜻
- 지정한 기준(\n)에 맞춰 자르려고 했는데 적당한 위치가 없어서 그냥 통으로 청크로 만들었음. 그랬더니 설정한 최대 크기보다 커졌다는 경고

③ 임베딩 & DB 저장

✓ Vector DB

- 텍스트를 벡터화(Embedding)하면서 저장이 동시에 진행
- 그러므로, VectorDB를 생성할 때, 임베딩 모델 지정

✓ Embedding Model

- text-embedding-3-small
 - 1,536 차원 벡터로 임베딩(청크 단위 벡터화)
 - 빠르고 강력한 성능 : 전문 검색, RAG 등에서 업계 표준처럼 사용 됨

```
from langchain_openai import OpenAIEmbeddings
embedding_model = OpenAIEmbeddings(model = "text-embedding-3-small")
```

③ 임베딩 & DB 저장

✓ 임베딩 벡터 확인해보기

```
# 임베딩 벡터
sample_text1 = "농촌 계몽운동은 한국 근대화의 중요한 과정이었다."
vector1 = embedding_model.embed_query(sample_text1)

# 벡터 길이 및 일부 값 확인
print(f"임베딩 벡터 길이: {len(vector1)}")
print(f"첫 10개 벡터 값: {vector1[:10]}")
```

임베딩 벡터 길이: 1536

첫 10개 벡터 값: [0.0703125, 0.043121337890625, 0.028289794921875, 0.05963134765625, ...]

③ 임베딩 & DB 저장

✓ Vector DB 생성

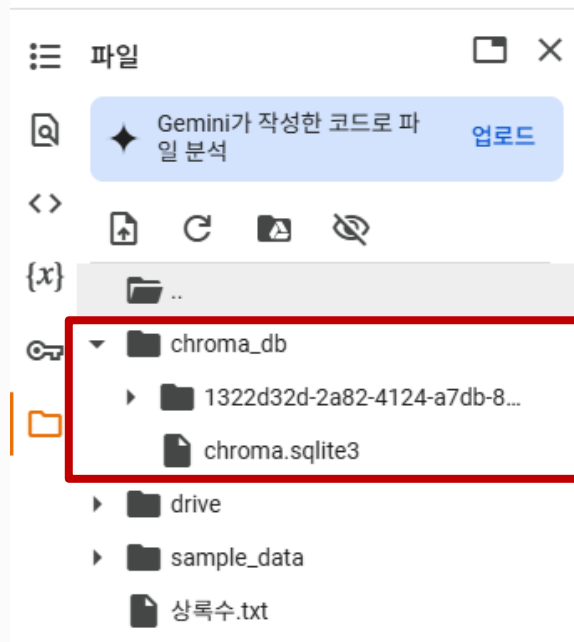
- 입력

- 청킹 리스트(split_texts)
- 임베딩 모델
- 저장위치 폴더

```
from langchain_community.vectorstores import Chroma

# ChromaDB를 만들면서 저장
vectorstore = Chroma.from_documents(split_docs, embedding_model,
                                     persist_directory="./chroma_db")
```

- Chroma.from_documents: 분할된 Document로 부터 DB 생성
- Chroma.from_texts: 분할된 텍스트로 부터 DB 생성



④ 벡터 DB 검색해보기

✓ DB에서 유사한 문장 검색

- .similarity_search : 유사도 상위 3개(k=3) 검색

```
query = "농촌 계몽운동에 대한 내용"
retrieved_docs = vectorstore.similarity_search(query, k=3)

# 결과 출력
for doc in retrieved_docs:
    print(doc.page_content)
    print('-'*200)
```

회원들이 그렇게 집안의 반대와 괴로움을 무릅쓰고 일을 하건만 실상 생기는 것이라고는 드러내어 말할 것이 못 된다. 공동답의 수확은 작년보다 대여섯 섬이나 늘었다. 개량식으로 지은 보람이 있어 재미가 나고...

하는 감상이 들었다. 그는 그러한 지도분자들과 굳게 악수를 하고 하룻밤씩 같이 자면서 의견을 교환하고 새로운 방침을 토론도 하였다. 어느 곳을 가나, "지금 우리의 형편으로는 계몽적인 문화운동도 해야 하지만, ...

S군은 저더러 잘 한다는 줄만 알고, 두번 세번 껴맞추고 나와서 익살을 깨트리는 바람에 점잔을 빼던 사회자도 간신히 웃음을 참고 앉았다. 그는 미소를 띠고 일어서며, ...

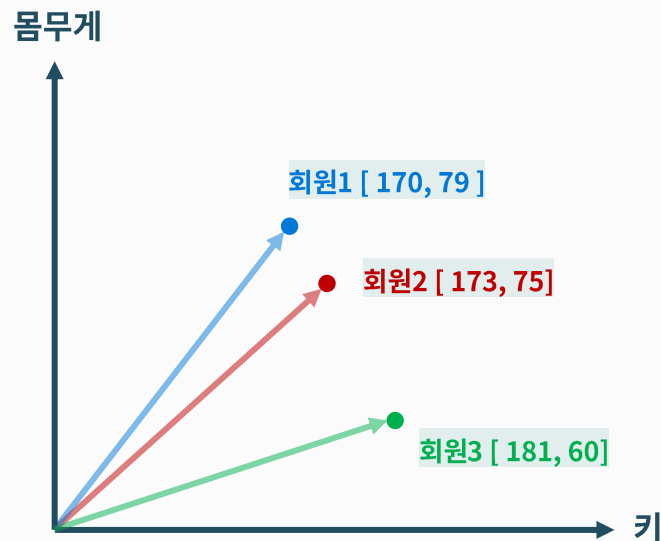
유사도

✓ 데이터 관점에서 벡터

- 여러 개의 숫자(feature)가 모여서 하나의 개념을 표현하는 단위
- 예 : 회원(객체)를 표현하는 특징 2가지 : 키와 몸무게
 - 회원1 : [170, 79]
 - 회원2 : [173, 75]
 - 회원3 : [181, 60]
 - 회원1과 회원2, 3 중 누구와 가깝나요?

✓ 특징 공간(Feature Space)

- 이렇게 feature 를 표현한 공간을 특징 공간이라고 부름
- 특징 공간은 벡터 공간의 일종



유사도 - 벡터 기초

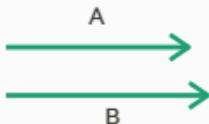
✓ 벡터의 내적(inner product)

- $a = [0.1, -0.8]$, $b = [0.9, 0.2]$
- $a \cdot b = (0.1 * 0.9) + (-0.8 * 0.2) = -0.07$
- 벡터의 길이 $\|a\| = \sqrt{(0.1^2 + (-0.8)^2)}$, 길이가 1인 벡터 : 단위 벡터

✓ $\cos \theta$

- 두 단위 벡터의 내적
- 값의 범위 : -1 ~ 1

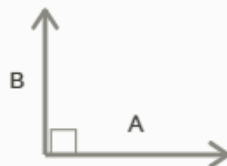
같은 방향 ($\theta = 0^\circ$)



내적 = 최대값 ✓

$\cos 0^\circ = 1$
→ 방향이 완전히 일치

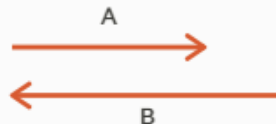
수직 ($\theta = 90^\circ$)



내적 = 0

$\cos 90^\circ = 0$
→ 아무 관련 없음

반대 방향 ($\theta = 180^\circ$)



내적 = 최소값 ✗

$\cos 180^\circ = -1$
→ 방향이 완전히 반대

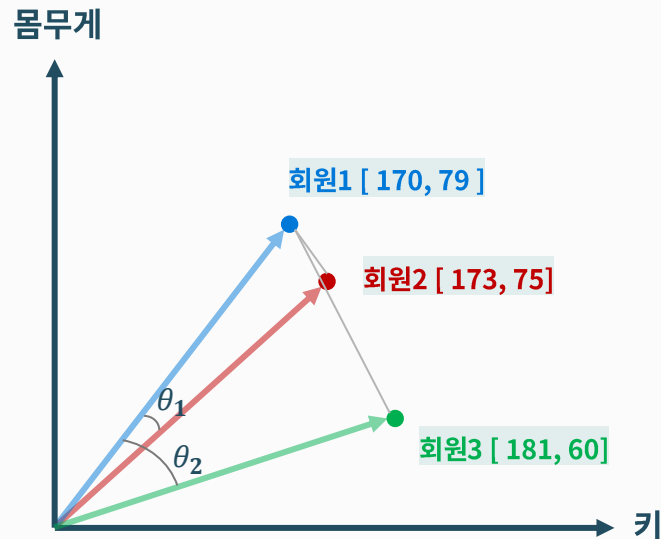
유사도

✓ 회원1,2,3의 유사도

- 회원1과 회원2는 가깝다.
- 회원1과 회원3은 멀다.

✓ 유사도 계산법 2개

- 사잇각으로 유사도 계산: $\cos \theta$ (코사인 유사도)
- 거리로 유사도 계산: 유클리드 거리(L2 유사도)

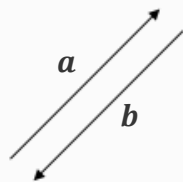


유사도

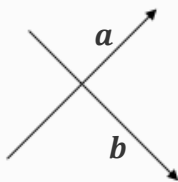
✓ **Cosine Similarity** $(a, b) = \cos \theta = \frac{a}{\|a\|} \cdot \frac{b}{\|b\|}$

a, b 는 벡터,
 $\|a\|$: 벡터 a 의 노름(*norm*, 길이, 크기)

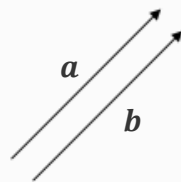
- 수학적 의미 : 두 단위 벡터의 내적
- NLP에서의 의미 : 유사한 벡터 → 유사한 문맥적 의미
- 1에 가까울수록 유사하고, 0이면 관계없고, -1이면 반대 의미



코사인 유사도 : -1



코사인 유사도 : 0



코사인 유사도 : 1

✓ **Cosine Distance**

- Cosine Distance $(a, b) = 1 - \text{Cosine Similarity}(a, b)$
- 0에 가까울수록 유사도가 높음

유사도

✓ L2 거리 (유클리드 거리)

- 두 벡터 간의 직선 거리
- 0에 가까울수록 유사, 클수록 다름
- 벡터의 크기와 방향 모두 반영

$$L2(a, b) = \sqrt{\sum (a_i - b_i)^2}$$

✓ L2 거리 vs 코사인 유사도 비교

항목	L2 거리	코사인 유사도
측정 기준	벡터 간 직선 거리	벡터 간 각도(방향)
계산 속도	빠름 (sqrt 생략하면 계산 비용 더욱 절감)	보통 (norm 계산 필요)

- L2 거리 계산시 정규화 되어 있으면 두 방법의 **순위 결과가 동일**

✓ Chroma DB에서의 유사도 점수 : $(L2\text{거리})^2$

④ 벡터 DB 검색해보기

✓ DB에서 유사한 문장 검색

- `.similarity_search_with_score`: 유사도 상위 3개(k=3) 검색, 유사도 점수 표시

```
query = "농촌 계몽운동에 대한 내용"
retrieved_docs = vectorstore.similarity_search_with_score(query, k=3)

for doc, score in retrieved_docs:
    print(f"Score(L2 ** 2): {score:.4f}")
    print(f"Content: {doc.page_content[:100]}")
    print("-" * 50)
```

Score(L2 ** 2): 1.3112

Content: 회원들이 그렇게 집안의 반대와 괴로움을 무릅쓰고 일을 하건만 실상 생기는 것이라고는 드러내어 말할 것이 못 된다. 공동답의 수확은 작년보다 대여섯 섬이나 늘었다. 개량식으로 지은

Score(L2 ** 2): 1.3324

Content: 하는 감상이 들었다. 그는 그러한 지도분자들과 굳게 악수를 하고 하룻밤씩 같이 자면서 의견을 교환하고 새로운 방침을 토론도 하였다. 어느 곳을 가나, "지금 우리의 형편으

Score(L2 ** 2): 1.3656

Content: S군은 저더러 잘 한다는 줄만 알고, 두 번 세 번 경충거리고 나와서 익살을 깨트리는 바람에 점잔을 빼던 사회자도 간신히 웃음을 참고 앉았다. 그는 미소를 띠고 일어서며,

| 벡터 DB 구축 : 다양한 방법

Vector DB 구성 : 다양한 방법

✓ Loader

- TXT Loader
- PDF Loader
- CSV Loader

✓ Splitter

- 로딩 후 만들어진 Document를 새로 조정된 단위로 분할 → Document 재구성

✓ Store

- Document 단위로 벡터화
- Chroma vs FAISS

Vector DB : Loader

✓ PDF Loader

- PDF를 로딩하여, 페이지별 하나의 Document 객체로 변환

```
from langchain_community.document_loaders import PyMuPDFLoader

# PDF 파일 로드
pdf_path = " 롯데쇼핑 서비스 이용약관.pdf"
pdf_loader = PyMuPDFLoader(path + pdf_path)

# 문서 로드 실행
doc_pdf = pdf_loader.load()

# 출력 확인
print(f"총 {len(doc_pdf)} 개의 페이지가 로드됨")
```

Vector DB : Loader

✓ CSV Loader

- 각 행을 하나의 Document 객체로 변환
- 예: 하나의 document
 - 칼럼1 : 내용
 - 칼럼2 : 내용

```
from langchain_community.document_loaders import CSVLoader

# CSV 파일 로드
csv_path = "sample.csv"
loader = CSVLoader(file_path=csv_path)

# 문서 로드 실행
documents = loader.load()
print(documents[0].page_content)
```

	구분	내용
0	낮선 라이선스 약관	현재 오픈소스 AI 라이선스 유형은 다양한 만큼 매우 복잡하다. 오픈소스 AI 모델...
1	낮선 라이선스 약관	메타와 유사하게 애플은 '애플 샘플 코드 라이선스'하에 오픈ELM을 출시했다. 이 ...
2	전문성 부족	오픈소스는 스스로 해야 하는 작업이 많을 수 있다. 기업은 코드를 다운로드할 수는 ...
3	전문성 부족	업계 전체에 전문성이 부족하다는 점은 새로운 문제를 야기한다. 원래 오픈소스의 장점...

구분: 낮선 라이선스 약관
내용: 현재 오픈소스 AI 라이선스 유형은 다양한 만큼 매우 복잡하다. 오픈소스 메타의 라마(Llama) 라이선스를 예로 보자. 라마는 높은 인기를 자랑하는 오픈

Vector DB : Splitter

✓ CSV로 로딩한 document

- 일반적으로 각 행을 청크의 의미로 보고, 별도 split 하지 않음
- 그러나, 만약 행의 정보가 길다면, split 해야할 수도 있음

	구분	내용	
0	낮선 라이선스 약관	현재 오픈소스 AI 라이선스 유형은 다양한 만큼 매우 복잡하다. 오픈소스 AI 모델...	청크1
1	낮선 라이선스 약관	메타와 유사하게 애플은 '애플 샘플 코드 라이선스'하에 오픈ELM을 출시했다. 이 ...	청크2
2	전문성 부족	오픈소스는 스스로 해야 하는 작업이 많을 수 있다. 기업은 코드를 다운로드할 수는 ...	
3	전문성 부족	업계 전체에 전문성이 부족하다는 점은 새로운 문제를 야기한다. 원래 오픈소스의 장점...	

소스 유형별 비교

✓ 다양한 소스로 부터 데이터 로딩 시, 다음과 같은 Document로 저장

소스 유형	Loading	Split
Text	TextLoader : 전체가 1개 Document로 로딩	CharacterTextSplitter : 글자, 문장, 문단 단위 분할
PDF	PyMuPDFLoader : 페이지 별 Document로 로딩	- CharacterTextSplitter : 글자, 문장, 문단 단위 분할 - 페이지 별 분할 → 로딩한 그대로 저장
CSV	CSVLoader : 행 별 Document로 로딩	행 단위 분할 → 로딩한 그대로 저장

- Document 단위로 임베딩 벡터 변환 및 저장됨

Vector DB : Embedding & Store

✓ .from_documents()

- 신규 DB 생성하면서, 문서 입력

✓ .add_documents()

- 기존 DB에 문서 추가

```
new_docs = [  
    Document(page_content="조선시대의 교육 제도는 성균관 중심이었다.", metadata={"source": "추가"}),  
    Document(page_content="한국 전통 사회에서 글을 읽는 능력은 권력의 상징이었다.", metadata={"source": "추가"})  
]  
  
# 문서 추가  
vectorstore.add_documents(new_docs)
```

Vector DB : Embedding & Store

✓ Vector DB 살펴보기

- 모든 문서 조회 : .get()
 - 저장된 모든 문서를 하나씩 추출하기

```
# ChromaDB에서 모든 데이터 가져오기
data = vectorstore.get()

# 결과 출력 (ids와 documents가 매칭되어 있습니다)
for i in range(len(data['ids'])):
    doc_id = data['ids'][i]
    content = data['documents'][i]
    metadata = data['metadatas'][i] if data['metadatas'] else {}

    print(f"[{i+1}] 문서 ID: {doc_id}")
    print(f"내용 요약: {content[:300]}..." ) # 300자 출력
    print(f"메타데이터: {metadata}")
    print("-" * 50)
```

Vector DB 살펴보기

✓ 전체 문서 수

```
print(db_pdf._collection.count())
```

✓ 전체 데이터 가져오기

```
# 전체 데이터 가져오기
result = db_pdf.get()

# 포함된 키: ids, documents, metadatas, embeddings(기본 제외)
print(result.keys())          # dict_keys(['ids', 'embeddings', 'documents', 'metadatas'])
print(result['ids'])           # 문서 ID 목록
print(result['documents'])     # 청크 텍스트 목록
print(result['metadatas'])     # 메타데이터 목록 (페이지 번호, 소스 파일 등)
```

Vector DB 살펴보기

✓ 조건 검색

- id로 조회

```
ids = result['ids'][:3]  
result = db_pdf.get(ids = ids)  
print(result['documents'])
```

- 메타 데이터로 필터링

```
result = db_pdf.get(where = {"page": 5})
```

- 특정 키 값 포함

```
result = db_pdf.get(include=["embeddings", "documents"], where={"page": 5})
```

요약 | Overview

✓ RAG란 : 별도 지식 DB를 구축하고, LLM이 답변 시 이를 참조하는 방식

- 사용자 입력 → 지식 검색(n개) → 사용자 입력 + 검색된 지식을 LLM에 전달 → LLM이 답변 생성

✓ RAG 구성 절차 : Load → Split(Chunking) → Embedding → Store

- Split : 지식 단위로 쪼개는 작업. 너무 크면 → Context 크기 증가(비용 ↑, 답변품질 ↓)
- Embedding : 지식 단위를 벡터화
- Store : 벡터 DB로 저장

✓ 지식 검색 : 벡터 유사도 기반 지식 검색

- 유사한 벡터일 수록 유사한 문맥적 의미를 갖게 됨
- 코사인 유사도(cosine similarity) : 1에 가까울수록 유사, 0이면 관계없고, -1이면 반대 의미
- L2 유사도 (유클리드 거리)
 - Chroma DB의 기본 유사도 계산법
 - 코사인 유사도 보다 계산 효율 높고(비용 낮고), 검색 순위는 동일

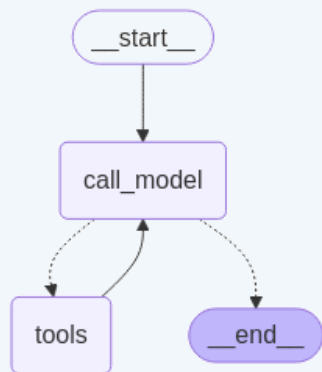
AI Agent with RAG

RAG를 Agent에 구현하기

✓ 다음과 같은 두가지 방법으로 RAG를 구현해 봅시다.

Case ① : Vector DB를 tool로 사용

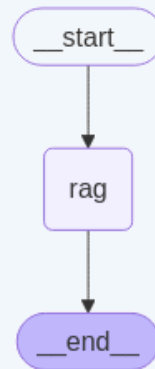
Vector DB를 tool 노드에 포함하여 Agent 구현



- 필요할 때만 RAG 호출(비용/성능 최적화)
- 잘못된 tool 호출 시 품질 저하 가능

Case ② : LLM with RAG를 하나의 노드로 구현

LLM with RAG를 하나의 노드로 Agent 구현



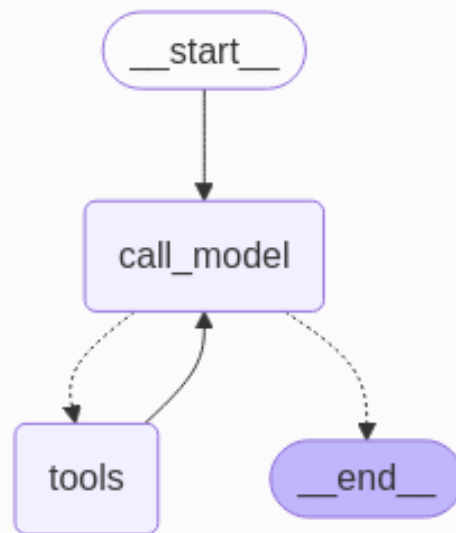
- 필요 여부 판단 없이 항상 RAG 검색 수행

| Case ① : Vector DB를 tool로 사용

Case ① : Vector DB를 Tool로 사용

✓ Vector DB를 도구(tool)로 사용

- GPT가 스스로 판단해서 검색할지 말지를 결정 : ReAct 스타일의 RAG 시스템
 - ReAct : Reasoning + Acting
 - LLM이 스스로 추론(Reasoning)
 - 필요한 경우 툴을 호출해서 행동(Acting) 하는 구조
- 구현 방식 : Vector DB를 tool 노드로 분리
- LLM 호출(call_model)과 tool 노드를 별도로 구분
- 확장성/제어 유리
 - 툴 추가 (예: 웹 검색, 계산기 등)
 - 복잡한 판단 로직 삽입 가능 (예: 분기 처리)



Case ① : Vector DB를 Tool로 사용

✓ 구축 절차

- (1) Vector DB 준비 : Vector DB 생성
- (2) Tool 정의 및 Tool Node 준비
 - Vector DB로 부터 검색하는 함수를 툴로 정의하고 노드 정의
- (3) LLM+Tool 바인딩
- (4) 노드 준비 및 그래프 구성

Case ① : Vector DB를 Tool로 사용

✓ Vector DB 준비 : Vector DB를 만들기

▪ 로딩 > 분할 > 임베딩 & 저장

```
# 지식 로딩
pdf_path = " 롯데쇼핑 서비스 이용약관.pdf"
pdf_loader = PyMuPDFLoader(path + pdf_path)
doc_pdf = pdf_loader.load()

# 분할
text_splitter = CharacterTextSplitter(
    chunk_size = 500,
    chunk_overlap = 100,
    separator = '\n', # \n 줄바꿈 문자 기준으로 자르기,
)
split_docs = text_splitter.split_documents(doc_pdf)

# 벡터 DB 정의
embedding = OpenAIEmbeddings(model="text-embedding-3-small")
db = Chroma.from_documents(split_docs, embedding, persist_directory="db01")
```

Case ① : Vector DB를 Tool로 사용

✓ Tool 정의 및 Tool Node 준비

- Vector DB로 부터 검색하는 함수를 툴로 정의하고 노드 준비
- 도구 함수 : 데코레이터 + Docstring

```
# 검색 함수 정의
@tool # 데코레이터
def rag_search_tool(query: str) -> str:
    "신용카드 약관 내용 검색시 사용" # docstring : 도구 설명

    docs = db.similarity_search(query, k=3)

    # 검색된 문서 내용들을 하나의 문자열로 반환
    return "\n\n".join(d.page_content for d in docs)

# tool 노드 준비
tools = [rag_search_tool]
tool_node = ToolNode(tools)
```

Case ① : Vector DB를 Tool로 사용

✓ LLM+Tool 바인딩

```
llm = ChatOpenAI(model="gpt-4.1-mini", temperature = 0.2).bind_tools(tools)
```

- RAG 기반의 Agent 구성 시 temperature 값 범위 예시

Agent 유형	권장 Temperature	이유
정보 검색 / QA 중심 Agent (RAG)	0.0 ~ 0.3	정답 기반 응답, 툴 사용 일관성 유지
ReAct 기반 판단 Agent	0.2 ~ 0.4	툴 사용 판단에서 안정성과 유연성 균형 필요
도움말/고객지원 Agent	0.3 ~ 0.5	약간의 유연성 허용 (동일 질문에 다른 말로 응답)
아이디어 생성 / 브레인스토밍 Agent	0.7 ~ 1.0	다양하고 창의적인 응답 필요

Case ① : Vector DB를 Tool로 사용

✓ 노드 준비

- 벡터 DB에 없는 내용에 대한 답변 생성 지침 필요

```
def chatbot(state: SimpleState):  
    # 1. State에서 필요한 정보 저장  
  
    # 2. prompt 구성  
    sys_msg = SystemMessage("""  
    # 역할  
    신용카드 고객 문의에 답변하는 에이전트야.  
  
    # 지침  
    - 사용자의 질문에 rag_search_tool을 사용해서 문서를 검색한 후 답변해줘.  
    - 문서에 없는 내용은 모른다고 말해.  
    """)  
  
    # 3. llm 호출  
    result = llm.invoke([sys_msg] + state["messages"])  
  
    # 4. state 업데이트  
    return {"messages": result, "response": result.content}
```


Case ① : Vector DB를 Tool로 사용

✓ 그래프 구성

- Tool 사용 여부에 따른 Conditional Edge 연결
 - 조건 분기 함수 : **tools_condition**

```
builder = StateGraph(SimpleState)

builder.add_node("chatbot", chatbot)
builder.add_node("tools", tool_node)           # RAG 도구 노드

builder.add_edge(START, "chatbot")
builder.add_conditional_edges("chatbot", tools_condition)  # tool_use → tools, 아니면 END
builder.add_edge("tools", "chatbot")           # 도구 실행 후 다시 chatbot 으로

graph = builder.compile()
```

Case ① : Vector DB를 Tool로 사용

✓ 실행

```
initial_state : SimpleState = {"messages": [HumanMessage("LangGraph는 무엇인가요?")]}
result = graph.invoke(initial_state)

for message in result["messages"]:
    message.pretty_print()
```

✓ Agent 동작 절차

- Human Message : 질문
- AI Message : 도구 호출
- Tool Message : 검색 결과
- AI Message
 - 검색 결과, 질문에 적절한 문서 없음
 - 답변 생성

```
===== Human Message =====
LangGraph는 무엇인가요?
===== AI Message =====

[{'text': 'LangGraph에 대해 검색해드리겠습니다.', 'type': 'text'},
 {'id': 'toolu_', 'caller': {'type': 'direct'}, 'input': {'query': 'LangGraph'}, 'name': 'rag_search_tool', 'type': 'tool_use'}]
Tool Calls:
  rag_search_tool (toolu_012HwW3wpjEtN1fjTDMXzMYC)
  Call ID: toolu_012HwW3wpjEtN1fjTDMXzMYC
  Args:
    query: LangGraph
===== Tool Message =====
Name: rag_search_tool

주요 사고 사례 및 동향
지난 3월, 리눅스 및 GNU 그룹에서 기본적으로 제공되는 데이터 압축 유틸리티(XZ Utils)의

2024년 사이버 위협 사례 분석
또한 최근 랜섬웨어는 피해 시스템 내부의 합법적인 자원(SW)을 공격에 악용하는

오용 가능성이 커진다. 이에 따라 기업들은 안전한 생성형 AI 사용을 위해 도입 단계에서 보안을

===== AI Message =====
죄송합니다. 검색 결과에서 LangGraph에 대한 정보를 찾을 수 없었습니다.
제가 접근할 수 있는 문서는 사이버 보안 관련 자료이며, LangGraph는 사이버 보안 위협 관련 문서에 포함되어 있지 않습니다.
```

| Case ② : LLM with RAG를 하나의 노드로 구현

Case ② : LLM with RAG를 하나의 노드로 구현

✓ 항상 검색 하고 검색 결과를 프롬프트에 포함

✓ 절차

- (1) Vector DB 준비
- (2) 벡터 DB 검색 함수 준비
- (3) 노드 구성
 - 검색 수행 및 결과를 프롬프트에 추가
 - LLM 답변
- 그래프 정의 : 벡터 DB 사용을 위한 조건 분기 필요하지 않음

Case ② : LLM with RAG를 하나의 노드로 구현

✓ 벡터 DB 검색 함수 준비

- 검색 결과를 하나의 문자열로 합쳐서 return

```
def rag_search(query: str):  
    # 검색  
    docs = db.similarity_search(query, k=3)  
  
    # 검색결과 합쳐서 return  
    return "\n\n".join(d.page_content for d in docs)
```

- 적정 k값(검색할 지식 수)은 Case by case
 - 청크(문서, 지식)가 크고, k값이 크면 프롬프트가 길어짐
 - 비용 증가
 - llm의 입력 범위 벗어날 수 있음
 - 너무 많은 내용은 정확한 답변을 생성하는 데에 오히려 방해가 될 수도 있음

Case ② : LLM with RAG를 하나의 노드로 구현

✓ 노드 구성

- 검색 수행 및 결과를 프롬프트에 추가
- LLM 답변

```
def rag_chatbot(state: SimpleState):  
    # 1. State에서 필요한 정보 저장  
    query = state["messages"][-1].content  
  
    # RAG 검색  
    search_result = rag_search(query)  
  
    # 2. prompt 구성  
    sys_msg = SystemMessage(f"""  
    # 역할  
    너는 사이버 보안 전문가야.  
  
    # 지침  
    - 사용자 질문과, 아래 검색 결과를 참조하여 답해  
    - 질문과 검색 결과가 일치 하지 않으면, 추론하지 말고 모른다고 답해  
  
    # 검색결과  
    {search_result}  
    """)
```

```
# 3. llm 호출  
result = llm.invoke([sys_msg] + state["messages"])  
  
# 4. state 업데이트  
return {"messages": result, "response": result.content}
```

요약 | Overview

✓ (1) Vector DB를 tool로 사용

- Vector DB를 하나의 tool로 지정해서 그래프 구성
- GPT가 스스로 판단해서 검색할지 말지를 결정 : **ReAct 스타일의 RAG 시스템**
- LLM 노드와 retriever(tool) 노드를 별도로 구분
 - 확장성/제어 유리 : 툴 추가 (예: 웹 검색, 계산기 등), 복잡한 판단 로직 삽입 가능 (예: 분기 처리)
 - 동적 사용 : 필요할 때만 RAG 호출 → 비용/성능 최적화

✓ (2) LLM with RAG를 하나의 노드로 구현

- 항상 검색한 후 결과를 기반으로 답변하도록 구성
- 구조 단순 : 하나의 노드로 처리 → 구현/디버깅 쉬움

종합실습 : 롯데월드 고객 응대 Agent v1.0

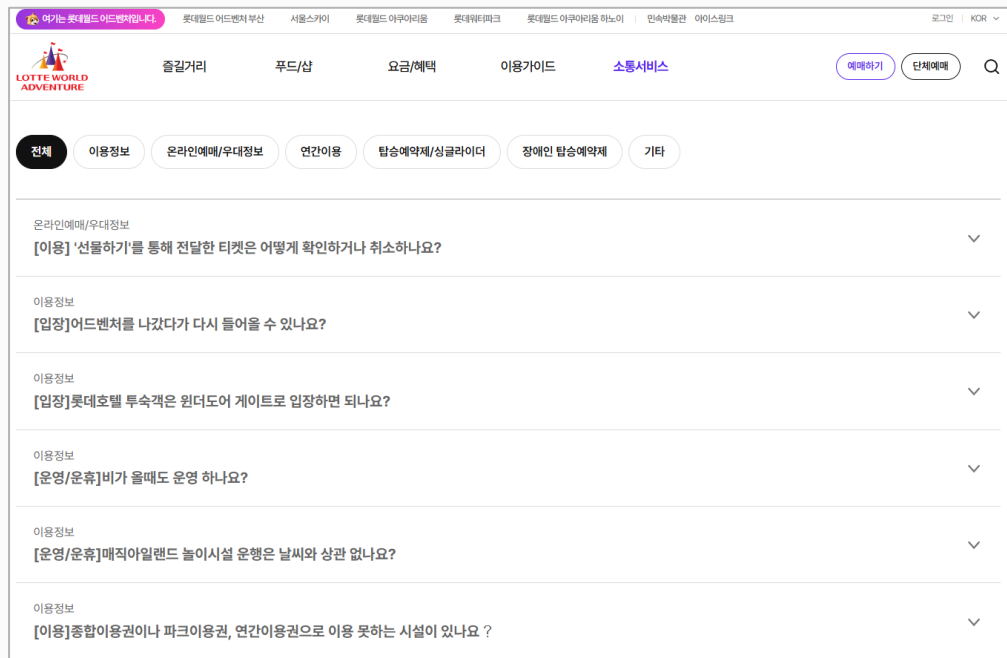
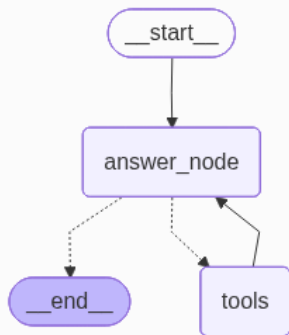
롯데월드 고객 응대 Agent 만들기

✓ 롯데월드 고객센터 홈페이지 : FAQ

<https://adventure.lotteworld.com/communication/faq/list?pageIndex=1&faqCd=&q=&linkType=>

✓ 1단계 : 기본 구성

- 도구 구성 : 벡터 DB, 웹 검색
 - 제공된 csv 파일에 FAQ 내용을 최대한 추가 (추가로 30건 이상)
- Agent : 도구를 활용한 답변 생성



롯데월드 고객 응대 Agent 만들기

✓ 2단계

- **Agent1 : 질문 답변 초안 작성**
 - 도구 참조
 - 고객 질문에 답변 초안 생성
- **Agent2 : 초안을 보완하여 최종 답변 생성**
 - Agent1의 답변 초안 검토
 - 답변 수정한 최종안 작성

✓ 3단계

- 어떻게 하면 좀 더 정확한 답변을 할 수 있을까요?
- 좀 더 고도화 해 봅시다.

